

Image Emotion Classification - Emoset 118k

Jakub Zaręba

[molab](#)

[colab](#)

Introduction

While traditional computer vision tasks involve objective classifications (e.g., "Is this a dog?"), identifying an emotion like "Awe" or "Contentment" from an image requires interpreting lighting, color palettes, contextual cues, and micro-expressions. This subjectivity creates an "affective gap" between pixel-level data and high-level human psychology.

The goal of this project was to build a machine learning pipeline capable of classifying images into distinct emotional categories. More importantly, the project aimed to test various architectures and training strategies while being bottlenecked by hardware constraints.

The Data

The first step was securing the right data. This project went through two distinct phases of dataset selection.

Attempt 1: The AAI-2016 FI Dataset

Initially, the pipeline was tested on the AAI-2016 Flickr and Instagram (FI) dataset. While academically relevant, the dataset was severely limited by extreme class imbalances and highly noisy human labeling. Training on this data within Google Colab and marimo molab quickly hit a performance ceiling, forcing a pivot.

Attempt 2: EmoSet-118K

To improve data quality and pipeline efficiency, the project migrated to the modern EmoSet-118K dataset, which features eight balanced classes (amusement, awe, contentment, excitement, anger, disgust, fear, and sadness).

Training on 118,000 images is too computationally expensive to run on notebooks with free GPU access, so to enable rapid prototyping, a 10% stratified subset was extracted. This subset was then split into an 80/10/10 (Train/Validation/Test) distribution.

Environment and Architectural Iterations

The initial experimental setup was hosted in Google Colab. However, strict compute limitations and memory constraints made training models on the AAI-2016 dataset impractical. To secure more compute, the environment was migrated to Molab to utilize a free NVIDIA RTX 6000 GPU.

Within Molab, initial attempts focused on training a high-resolution Vision Transformer (ViT-Base) from scratch, without pre-trained weights. While this model showed learning progress after 12 to 15 epochs, the computational cost was prohibitive, requiring 150 to 200 seconds per epoch. This slow feedback loop severely bottlenecked hyperparameter tuning and model evaluation.

To resolve this, the project was restructured. The AAAI dataset was abandoned in favor of a 10% subset of the EmoSet-118K dataset, and the model architecture was scaled down to `vit_small_patch16_224`. This architectural shift along with torch optimizations reduced training time to approximately 1 minute per epoch, allowing a quick 8-epoch run to complete in under 10 minutes. This accelerated iteration speed became the foundation for all following optimization.

Training Methodology

While scaling down to `vit_small` drastically improved training speed, the initial accuracy plateaued at a maximum of roughly 45%.

To address this, pre-trained ImageNet weights were introduced alongside a two-phase training:

Phase 1: Warmup: For the initial epochs, the pre-trained backbone is entirely frozen. Only the newly initialized classification head is trained. This initializes the head with appropriate weights without degrading the foundational ImageNet features via backward propagation.

Phase 2: Fine-Tuning: The backbone is unfrozen, and differential learning rates are applied. The classification head continues updating at the standard rate, while the backbone is updated at a reduced rate (10% of the base learning rate) to gently adapt the feature maps to the specific emotion classes.

Implementing this pre-training and warmup strategy immediately increased model accuracy from 45% to approximately 70%.

With a viable 10-minute training loop established, the pipeline was automated within a reactive Marimo notebook UI. Interactive sliders and dropdowns were added to easily manipulate batch size, learning rate, weight decay, image input size and warmup epochs without rewriting code. This setup enabled rapid testing across seven distinct architectures: `vit_small`, `vit_base`, `convnext_tiny`, `convnext_base`, `efficientnetv2_rw_s`, `efficientnetv2_rw_m`, and `tf_efficientnetv2_l`. Small and tiny models were almost exclusively prioritized to maximize the number of iterations.

Optimization and Hardware Efficiency

To maximize the utilization of the RTX 6000 GPU and maintain fast iteration speeds, several optimization components were implemented into the PyTorch training loop:

Optimizer: AdamW to update the model's weights as it learns. It includes weight decay that stops the model from simply memorizing our small dataset, forcing it to actually learn the underlying emotional patterns.

Scheduler: A Cosine Annealing scheduler to control how fast the model learns over time. It smoothly slows down the learning speed as training progresses, allowing the model to make finer, more precise adjustments as it gets closer to the finish line.

Scaler: Automatic Mixed Precision (AMP) to speed up calculations and save GPU memory. It automatically switches to smaller, faster number formats allowing the GPU to work faster without crashing the training process.

JIT Compilation: `torch.compile` to figure out the most efficient way to run the model's calculations before it even starts. It saves this optimized plan, making every round of training after the first one noticeably faster.

Evaluation Metrics

A critical step to measure how well the model actually performs on unseen data. The pipeline evaluates the model against the 10% test split using the following metrics:

Test Set Accuracy: The overall percentage of times the model predicted the correct emotion across all images in the test group.

Precision, Recall, and F1-Score: Because overall accuracy can be misleading, we break down performance for each of the eight distinct emotions.

Results

After testing various architectures, `convnext_tiny` was selected as the final model. It provided the best balance between high-level feature extraction and computational efficiency, ensuring the training pipeline could run reliably within Google Colab's constraints.

The final training run utilized a batch size of 128, a learning rate of 1e-4, and a weight decay of 0.1. The model was trained over 10 epochs using the two-phase strategy (2 warmup epochs with a frozen backbone, followed by 8 epochs of fine-tuning).

Test Set Evaluation

When evaluated against the unseen test split (945 images), the model achieved a final overall accuracy of 75%.

A breakdown of the per-class performance reveals how the "affective gap" impacts the model's predictive capabilities differently across various emotions:

Emotion	Precision	Recall	F1-Score	Support (Images)
Disgust	0.85	0.83	0.84	84
Excitement	0.80	0.84	0.82	158
Anger	0.80	0.82	0.81	85
Awe	0.79	0.75	0.77	120
Fear	0.77	0.76	0.77	108
Sadness	0.74	0.71	0.72	102
Amusement	0.71	0.69	0.70	157
Contentment	0.59	0.64	0.62	131
Macro Avg	0.76	0.75	0.76	